

Chapter 58: Production Bugs

Scenario 1: The Dashboard That Stops Updating After the First Load

Situation: A live trading dashboard shows real-time price ticks pushed over a WebSocket. QA reports that prices update fine for the first 30-60 seconds, then the numbers on screen just freeze — even though the network tab shows messages still arriving every second. Refreshing the page fixes it temporarily, then it freezes again. It happens on every machine, not just slow ones.

Question: How do you diagnose and fix a component that appears to stop reacting to incoming data despite the data clearly still arriving?

Answer:

First step: confirm whether this is a "data isn't arriving" problem or a "data arrives but view doesn't update" problem. The network tab already tells us messages keep arriving, so the bug is in change detection, not the data source.

Next, check how the WebSocket callback updates the component's state. A classic cause: the WebSocket client is a raw browser API (not an Angular-wrapped one), so its `onmessage` callback runs **outside the Angular zone** only if someone has explicitly called `runOutsideAngular` for it (common for perf reasons on high-frequency streams) — but then whoever wrote the assignment forgot to bring it back inside the zone or to manually trigger detection. In older code this looked like:

```
this.ngZone.runOutsideAngular(() => {
  this.socket.onmessage = (msg) => {
    this.price = JSON.parse(msg.data).price; // never triggers CD
  };
});
```

This would explain "runs forever but view frozen" — not "freezes after 30-60s" though. The time-boxed freeze detail is the misleading clue: it suggests something that degrades gradually, not a static wiring bug. That points to zone starvation from a **queued microtask/interval leak**: e.g., a `setInterval` elsewhere (a "connection health" ping) that throws silently inside the zone, or an RxJS subscription chain using `share()` / `shareReplay()` without `refCount`, where an unhandled error in one subscriber tears down the shared source for everyone after N ticks, but the socket keeps emitting into a dead pipe.

Diagnosis steps:

1. Add `console.count('tick')` inside the actual message handler to prove data is reaching the component method itself (not just the socket).
2. If the handler is invoked but the DOM doesn't change, log `NgZone.isStable` or check `ApplicationRef.isStable` in devtools — if it flips to stable and stays stable while data still arrives, CD truly stopped being scheduled.
3. Check the console for a swallowed error — RxJS silently completes/errors a shared observable if a `catchError` wasn't applied per-subscriber.

Root cause found in this real case: a `shareReplay(1)` pipe with no `catchError`, combined with a downstream consumer that threw a `TypeError` on a null field roughly 30-60 messages in (a field that's occasionally absent in the price payload). The error unsubscribed everyone from the shared source, but the raw socket's `.next()` calls into a completed Subject are no-ops — hence "frozen" not "errored."

Fix: make the stream resilient to per-message errors and use signals so we don't depend on zone-triggered CD at all:

```
private price = signal<number | null>(null);
readonly displayPrice = computed(() => this.price() ?? 0);

private priceStream = this.wsservice.messages$.pipe(
  map(raw => this.parsePrice(raw)),
  catchError((err, caught) => {
    console.error('bad tick, skipping', err);
    return caught; // resubscribe to the same source, don't kill it
  }),
  takeUntilDestroyed()
).subscribe(price => this.price.set(price));
```

Verification: replay the same malformed payload in a test harness and confirm the stream survives and keeps updating. Add a Sentry/console alert on any `catchError` hit so the next null-field regression is visible immediately instead of silently freezing the UI.

Interviewer intent: Tests whether the candidate treats "stopped updating" as a change-detection problem first, then correctly reads misleading timing clues to find a shared-observable error-swallowing bug rather than jumping straight to "it's a zone.js issue."

Scenario 2: The Tab That Gets Slower the Longer It's Open

Situation: Support tickets say the internal admin tool becomes "laggy" after being left open for a few hours — typing in search boxes lags, tab switching stutters. Task Manager shows the tab's memory climbing steadily from 150MB to over 1.5GB over a workday. Closing and reopening fixes it. It happens more on the "Reports" screen where users tend to leave the tab open and switch between report tabs frequently.

Question: Walk through how you'd track down and fix this memory leak.

Answer: This is the textbook "the tab slows down over time" memory leak, and the fact that it correlates with the Reports screen and switching between report tabs is the key lead — it implicates a component that is created and destroyed repeatedly (each report tab) rather than a single long-lived one.

Diagnostic steps:

1. Open Chrome DevTools → Memory tab, take a heap snapshot, switch between report tabs 20 times, take another snapshot, and use "Comparison" view to see what object counts grew. Look specifically for retained instances of the report component class and any `Subscription` / `EventEmitter` / `Subject` objects.
2. If the component instances themselves are retained (not garbage collected) after navigating away, that means something is holding a reference — almost always an **unsubscribed subscription to a long-lived service**, a **DOM event listener registered on `window` / `document` and never removed**, or a

third-party library instance (e.g., a charting library) that keeps an internal reference to the host element/component.

The actual root cause in this case: the Reports component subscribes to a shared `dataRefreshService.updates$` (a global interval-based Subject) in `ngOnInit` without unsubscribing in `ngOnDestroy`:

```
// BEFORE – leaks one subscription per component instantiation
ngOnInit() {
  this.refreshService.updates$.subscribe(() => this.reload());
}
```

Every time a user opens a report tab, a new subscription is created; the global Subject holds a reference to the callback closure, which holds a reference to `this` (the component), so the component and its entire view tree can never be garbage collected. Over a workday of dozens of tab switches, dozens of dead components stay alive, each running `reload()` on every tick — which also explains the growing CPU lag, not just memory.

Fix — always tie subscriptions to the component lifecycle, preferably via `takeUntilDestroyed()`:

```
export class ReportComponent {
  private refreshService = inject(RefreshService);
  private destroyRef = inject(DestroyRef);

  constructor() {
    this.refreshService.updates$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(() => this.reload());
  }
}
```

Also add a lint rule (`rxjs-angular/prefer-takeuntil` or a custom ESLint rule) to catch bare `.subscribe()` calls on injected service observables without a teardown operator in code review going forward.

Verification: repeat the heap snapshot comparison after the fix — object count for the report component class should return to zero after navigating away, confirmed via `retainers` view showing no path from GC roots.

Interviewer intent: Confirms the candidate can use Chrome DevTools heap snapshots methodically (not just "add `takeUntilDestroyed` everywhere") and understands *why* a subscription to a long-lived service is what prevents component garbage collection.

Scenario 3: The Save Button That Creates Duplicate Records

Situation: Customers occasionally end up with two identical orders after clicking "Place Order" once. Support initially blamed users for double-clicking, but session recordings show some users click exactly once, wait, and still get two records. It's intermittent and worse on slower connections/mobile networks.

Question: Diagnose this race condition and propose a fix that also protects against the "double click" case, not just the single-click network case.

Answer: The session recordings ruling out double-clicks are the crucial clue: this isn't UI-level debouncing failure, it's a race between **the button being clickable again before the in-flight request resolves**, likely combined with **retry behavior** somewhere in the HTTP pipeline (an interceptor retrying on timeout, or the browser/service worker retrying a POST after a flaky connection).

Diagnosis steps:

1. Reproduce with network throttling (Slow 3G) and check the Network tab for the exact number of POST requests fired per single click.
2. Check for an `HttpInterceptor` using `retry()` / `retryWhen()` on all requests indiscriminately — a common mistake is applying a blanket retry policy meant for idempotent GETs to POST/PUT as well.
3. Check the click handler itself for missing disabled-state guarding during the async call.

In this case, both problems existed together: a global interceptor retried any request that took longer than a timeout threshold (meant to paper over flaky mobile networks), and the submit handler didn't disable the button or guard against concurrent invocation:

```
// BEFORE
onSubmit() {
  this.orderService.placeOrder(this.form.value).subscribe(res => {
    this.router.navigate(['/confirmation', res.id]);
  });
}
```

```
// interceptor - retries a non-idempotent POST, causing duplicate submits on slow networks
intercept(req: HttpRequest<any>, next: HttpHandler) {
  return next.handle(req).pipe(retry(2));
}
```

Fix has three layers, because relying on only one is fragile:

```
// 1. Exclude non-idempotent methods from blanket retry
intercept(req: HttpRequest<any>, next: HttpHandler) {
  if (req.method !== 'GET') return next.handle(req);
  return next.handle(req).pipe(retry(2));
}
```

```
// 2. Guard the click handler with an in-flight flag / disable the control
protected submitting = signal(false);

onSubmit() {
  if (this.submitting()) return;
  this.submitting.set(true);

  this.orderService.placeOrder(this.form.value)
    .pipe(finally(() => this.submitting.set(false)))
    .subscribe({
      next: res => this.router.navigate(['/confirmation', res.id]),
      error: () => this.toast.error('Order failed, please retry.')
    });
}
```

```
}
```

```
<button [disabled]="submitting()" (click)="onSubmit()">
  {{ submitting() ? 'Placing order...' : 'Place Order' }}
</button>
```

```
// 3. Server-side idempotency key as the real safety net – client-side guards
// can't cover every case (e.g. tab duplication, replayed requests).
placeOrder(order: OrderPayload) {
  const idempotencyKey = crypto.randomUUID();
  return this.http.post('/api/orders', order, {
    headers: { 'Idempotency-Key': idempotencyKey }
  });
}
```

Verification: throttle network to Slow 3G, click once, confirm only one POST with a stable idempotency key is sent even if the interceptor or network causes a resend; the server rejects duplicate idempotency keys with a 200 returning the original record.

Interviewer intent: Tests whether the candidate defends in depth (client guard + interceptor scoping + server idempotency) rather than treating "disable the button" as a complete fix, since client-only fixes can't cover every race.

Scenario 4: The Multi-Step Form That Silently Loses Data on Tab Switch

Situation: Users filling out a long onboarding wizard (5 steps, each step a separate route with its own component) complain that if they switch to another browser tab to copy a document number and switch back, data they'd entered in earlier steps is gone when they finally submit. No errors in the console. It doesn't happen if they stay on the same browser tab the whole time.

Question: How would you investigate a form that "loses data" with no visible error, and what's the likely fix?

Answer: "No console errors" plus "only when switching tabs" strongly suggests this isn't a crash — it's a **state storage lifetime problem** tied to something that resets when the tab loses/regains focus or, more likely, when Angular's router destroys and recreates components as the user navigates between wizard steps, and the shared state was never actually being persisted anywhere durable.

Diagnosis steps:

1. Reproduce without switching tabs at all — just navigate forward through the 5 steps and back. If data is already lost at that point, tab-switching is a red herring and the real bug is component-scoped state being wiped on route change.
2. Check where the wizard's data lives: is it a `@Input()` / local field on each step component (destroyed on navigation), a service-level `signal` / `BehaviorSubject` provided at the right scope, or actually persisted to `sessionStorage` / a draft API call?
3. Check the DI provider scope of the shared "wizard state" service — a very common bug is providing it in the **route's component providers array** instead of a parent route or `providedIn: 'root'`. If each

step is its own routed component and the service is provided per-component, Angular creates a **new instance of the service per step**, silently discarding whatever was in the previous instance.

```
// BEFORE – service provided at the leaf route component level
@Component({
  selector: 'app-step-two',
  providers: [WizardDataService], // new instance every time this component is (re)created
  ...
})
export class StepTwoComponent {}
```

Because each step re-provides `WizardDataService`, step 1's answers live in an instance that gets destroyed the moment the user navigates to step 2. It "feels" like tab switching triggers it because switching away and back is often when users notice — the actual navigation between steps is the real cause, and it was losing data on every step transition already, tab or no tab.

Fix: provide the state service once at a scope that outlives all the child steps — either at the parent wizard route/component, or `providedIn: 'root'` if the flow shouldn't be nested under a parent route, and back it with `sessionStorage` so a hard refresh doesn't lose it either:

```
@Injectable({ providedIn: 'root' })
export class WizardDataService {
  private state = signal<WizardData>(this.restore());

  readonly data = this.state.asReadonly();

  update(patch: Partial<WizardData>) {
    const next = { ...this.state(), ...patch };
    this.state.set(next);
    sessionStorage.setItem('wizard-draft', JSON.stringify(next));
  }

  private restore(): WizardData {
    return JSON.parse(sessionStorage.getItem('wizard-draft') ?? '{}');
  }
}
```

Verification: fill step 1, navigate to step 5 directly via the stepper (not just Next), refresh the page, and confirm all prior answers are intact. Add an E2E test that walks the wizard forward/back/refresh in sequence so this regression is caught before release.

Interviewer intent: Tests understanding of Angular DI provider scoping and hierarchical injectors — a subtle bug class where "silently loses data" is really "silently creates a new service instance."

Scenario 5: The List That Flickers on Every Keystroke in a Search Box

Situation: A product search page shows a filtered list below a search input. As users type, the whole list visibly flickers/blinks on every keystroke — items seem to disappear and redraw, and on slower machines you can see a blank flash. The data itself is correct; it's purely visual.

Question: Diagnose the flicker and explain the fix.

Answer: A pure-visual flicker with correct data almost always means the list is being **completely torn down and rebuilt** in the DOM rather than being diffed/patched, which is a `@for / *ngFor trackBy` problem, or the filtering logic is producing entirely new object references every time so Angular's default identity-based reconciliation can't match old items to new ones.

Diagnosis steps:

1. Open the Elements panel, filter to highlight repaints (Rendering tab → "Paint flashing"), and type — confirm the whole `<ul>` subtree repaints instead of just text nodes.
2. Check the `@for` block for a `track` expression. If it's missing or uses something that changes every time (like `track $index` when the list order changes, or no track at all defaulting to identity), Angular will destroy and recreate every DOM node whenever the array reference changes, even if 95% of the items are the same objects.
3. Check the filter function — if it does `this.items.map(i => ({...i}))` or similar even when unfiltered, every keystroke produces brand-new object identities, defeating any track-by-id strategy too.

```
// BEFORE - no meaningful track key, plus new objects every filter pass
@for (item of filteredItems(); track $index) { ... }

filteredItems = computed(() =>
  this.items().filter(i => i.name.includes(this.query())).map(i => ({ ...i })))
);
```

Using `$index` as the track key is the core problem here: when filtering removes/reorders items, index 3 might now point to a completely different item than before, so Angular thinks "item at index 3 changed" and tears down/rebuilds that DOM node and every node after it, instead of recognizing that most items are literally unchanged and just need to be shown/hidden.

Fix: track by stable identity (the item's own id) and stop cloning objects unnecessarily:

```
@for (item of filteredItems(); track item.id) {
  <app-product-card [product]="item" />
}

filteredItems = computed(() =>
  this.items().filter(i => i.name.toLowerCase().includes(this.query().toLowerCase())))
);
```

Also worth checking: is the search input itself debounced? Filtering on every single keystroke without debounce isn't the root flicker cause here (that's the trackBy), but it compounds the perceived jank on large lists:

```
protected query = signal('');
private query$ = toObservable(this.query).pipe(debounceTime(150), distinctUntilChanged());
```

Verification: with Paint Flashing enabled, typing should now only highlight the cards whose visibility actually changed, not the entire list; profile with the Performance tab to confirm layout/paint time per keystroke dropped significantly.

Interviewer intent: Tests whether the candidate connects a purely visual symptom (flicker) to Angular's DOM reconciliation strategy and understands `track` semantics beyond "just add `trackBy` because the linter said so."

Scenario 6: The Bug That Only Reproduces in the Production Build

Situation: Everything works perfectly with `ng serve` in dev mode. After deploying a production build (`ng build --configuration production`), users report that clicking certain buttons throws a blank white screen with no visible error, though the app "was tested thoroughly" in dev and staging (which apparently runs a dev build).

Question: Why would a bug appear only in a production build, and how do you track it down?

Answer: The single biggest difference between dev and prod builds that causes "works in dev, breaks in prod" bugs is **minification/property renaming** combined with a few Angular-specific production-only behaviors: AOT compilation being stricter, and dev-mode-only checks being stripped out. Since staging apparently runs a dev-mode build, this bug has simply never been exercised against real production settings before.

Diagnosis steps:

1. Reproduce locally with `ng build --configuration production` and `ng serve --configuration production` (or serve the `dist` folder with `http-server`) rather than trusting staging.
2. Open the browser console on the broken production build — even minified, Angular/Zone errors usually still surface a stack trace; look for errors like `Cannot read properties of undefined (reading 'x')` where `x` is a single mangled letter, which is a strong signal of **property renaming breaking a dynamic property access** (e.g., `obj[someStringVariable]` where the string was hardcoded to a property name that gets minified away, or JSON parsing matched against a class shape that no longer has matching names post-minification).
3. Alternatively, check for reliance on function/class **names** at runtime — e.g., `error.constructor.name === 'HttpErrorResponse'` or logic branching on `someFn.name`, which breaks under minification since names get mangled/shortened in prod builds but not dev.
4. Check for **template type-checking differences**: production builds with AOT can catch template errors that JIT sometimes tolerated, though this usually fails the build rather than at runtime — still worth confirming the build genuinely succeeded without suppressed warnings.

The actual root cause found: a service does `Object.keys(response).includes('errorCode')` to decide how to handle an API error shape, but a *different* part of the code destructures the response using a renamed local alias matched against the literal string `'errorCode'` obtained via `Object.keys(SomeInterface)` reflection-style trick on a class instance rather than a plain interface — property renaming in prod (via `terser`) doesn't touch object literal string keys from JSON, but it does rename **class field names** if `--build-optimizer` / `terser`'s property mangling touches a class incorrectly relied upon for its field names via reflection. More commonly in real incidents, the actual culprit is simpler: someone used `instanceof` against a class from a **lazy-loaded chunk** that, under differential loading/module federation, ends up as a *different* evaluated class reference than the one in the main bundle, so `instanceof` fails silently only in the chunked production build.


```
// BEFORE – instanceof across a lazy chunk boundary, fragile in prod (differently bundled)
code paths
if (error instanceof HttpResponse) { ... }
```

The robust fix avoids `instanceof`/name-based checks across bundle boundaries and instead checks a duck-typed shape or Angular's own type guard, and — critically — sets up **production source maps** so future prod-only errors are actually readable:

```
// angular.json – enable source maps for the production build so stack traces are legible
"production": {
  "sourceMap": true,
  "namedChunks": false,
  "optimization": true
}
```

```
function isHttpResponse(err: unknown): err is HttpResponse {
  return !!err && typeof err === 'object' && 'status' in err && 'error' in err;
}
```

Verification: rebuild for production, upload source maps to the error monitoring tool (Sentry supports this directly), reproduce the click, and confirm the readable stack trace now points at the exact failing line instead of a minified blob. Going forward, staging must run the actual production build configuration, not a dev build, to catch this class of bug before release.

Interviewer intent: Tests whether the candidate knows the concrete mechanical differences between Angular dev and prod builds (minification, AOT strictness, tree-shaking, bundle splitting) rather than shrugging with "it's flaky."

Scenario 7: The Third-Party Chat Widget That Breaks Angular's Rendering

Situation: After marketing adds a third-party live-chat widget script (loaded via a `<script>` tag in `index.html`), users start reporting that some pages become unresponsive — clicking buttons does nothing, though the chat widget itself works fine. This only happens on pages that also have the chat widget visible; other pages are unaffected. Removing the widget script "fixes" it immediately.

Question: How do you diagnose interference between a third-party script and Angular's own event/rendering pipeline, and how do you contain it without removing the widget entirely?

Answer: The core suspicion with any third-party script that "works but breaks Angular" is that it's **monkey-patching global browser APIs** that Zone.js also patches — most commonly `addEventListener`, `setTimeout`, `Promise`, or `XMLHttpRequest` / `fetch` — and the two patching layers conflict, or the widget is attaching listeners in a way that stops propagation before Angular's own delegated handlers ever see the click.

Diagnosis steps:

1. Check script load order in `index.html`: if the third-party script loads and patches `addEventListener` / `Promise` *after* Zone.js has already patched them, Zone.js's patched versions get further wrapped or bypassed depending on how the widget saves references, and Angular's change

detection scheduling (which relies on those patches to know "something happened, run a CD cycle") can stop firing for events the widget's patch swallows.

2. In DevTools console, run `window.Zone` and confirm it's still the Zone.js instance Angular expects; check `NgZone.isStable` — if it goes stable and never leaves stable when clicking buttons on affected pages, click handlers are running fully outside Angular's zone (or not running at all because the widget's capturing-phase listener calls `stopPropagation()` before Angular's handler executes).
3. Use the Elements panel's "Event Listeners" tab on a broken button to see if the widget attached a global capture-phase listener on `document` that unconditionally calls `event.stopPropagation() / preventDefault()` for its own drag-handling logic, inadvertently swallowing clicks anywhere on the page while the widget is mounted.

Root cause identified: the widget attaches a `document.addEventListener('click', handler, true)`

(capture phase) globally to detect "click outside the chat bubble to minimize it" and calls

`event.stopPropagation()` unconditionally instead of only when the click is outside its own container.

Because it's registered in the *capture* phase, it runs before Angular's own bound `(click)` handlers (which are typically bubble-phase), so `stopPropagation()` there prevents the event from ever reaching Angular's listeners.

Fix options, from most to least preferred:

1. Ask the vendor/config for a "click outside" option scoped to the widget's own root element rather than swallowing global clicks — the correct real fix belongs in the third-party config.
2. If not configurable, load the widget script isolated in an iframe so its DOM/event manipulation can't touch the host page at all — the most robust containment.
3. As a stopgap, patch around it defensively by not relying on native click bubbling for critical actions — route key interactions through Angular's `HostListener` on the capture phase too, or use signals plus a dedicated global click service that both the app and any embedded widget-safe wrapper coordinate against.

```
// Defensive stopgap: run our own capture-phase listener that runs before the
// widget's, ensuring Angular still reacts even if a later listener stops propagation.
constructor() {
  const zone = inject(NgZone);
  zone.runOutsideAngular(() => {
    document.addEventListener('click', (e) => {
      const target = e.target as HTMLElement;
      if (target.closest('[data-app-action]')) {
        zone.run(() => this.handleAppClick(target));
      }
    }, true); // capture phase, registered before the widget's script runs
  });
}
```

Verification: with the widget present, click every primary action button across affected pages and confirm they still register; check `NgZone.isStable` toggles correctly around clicks. Long-term, isolate all third-party embeds in a sandboxed iframe as a standing policy so this class of interference can't recur with future vendor scripts.

Interviewer intent: Tests real-world debugging of Zone.js/DOM event conflicts with third-party code — a bug class candidates rarely encounter in tutorials but is extremely common in production apps that embed vendor widgets.

Scenario 8: Stale Data Displayed After Navigating Back

Situation: A user edits their profile, saves successfully (confirmed by a success toast and a fresh GET response in the network tab), navigates to the dashboard, then clicks back to the profile page — and sees the *old*, pre-edit data for a split second to several seconds before it "catches up." On slow connections the stale data sometimes just stays until a manual refresh.

Question: What's causing stale data to reappear after navigation, and how do you fix it, especially the case where it *never* catches up?

Answer: "Confirmed fresh GET in the network tab on save" tells us the backend and the save flow are fine — the bug is purely in **how the profile component sources its data on the return visit**. Two very common causes: (1) the component uses `resolve` guards or an `ngOnInit` fetch that re-requests data, but a **route reuse strategy** or Angular's default behavior for reused routes with the same route config causes the component to not be recreated, so `ngOnInit` never re-runs and the view shows whatever was in memory from before; or (2) the data is served from an HTTP cache (custom interceptor cache, or `shareReplay`-based service cache) that wasn't invalidated on save.

Diagnosis steps:

1. Add a `console.log` in `ngOnInit` of the profile component and navigate away/back — if it doesn't log at all on return, the component instance is being reused rather than recreated, which is a router configuration/reuse-strategy issue, not a data issue.
2. If `ngOnInit` *does* re-run, check the service method it calls — if it's backed by `shareReplay(1)` or a manual cache `Map` keyed by user id, confirm whether the save operation calls any cache-invalidation method afterward.
3. Check whether there's a custom `RouteReuseStrategy` in the app (common in apps with tab-like navigation to preserve scroll position) that's being too aggressive and reusing the profile route component even when its resolved data should be considered stale.

Root cause in this case: a `UserService` caches the current user via `shareReplay(1)` for legitimate reasons (avoid refetching on every component that needs the user), but the profile save flow calls a *separate* `updateProfile()` endpoint directly without going through the same service method, so the cached `shareReplay` observable never learns the data changed:

```
// BEFORE - cache never invalidated after a mutation
@Injectable({ providedIn: 'root' })
export class UserService {
  private user$ = this.http.get<User>('/api/me').pipe(shareReplay(1));
  getCurrentUser() { return this.user$; }
}

// profile.component.ts
save() {
  this.http.put('/api/me', this.form.value).subscribe(() => {
    this.toast.success('saved');
  });
}
```

```

    this.router.navigate(['/dashboard']);
  });
  // UserService.user$ still holds the old cached value
}

```

Fix: give the cache an explicit invalidation/refresh mechanism and route all reads/writes through the same service so state stays consistent:

```

@Injectable({ providedIn: 'root' })
export class UserService {
  private refresh$ = new BehaviorSubject<void>(undefined);

  readonly user$ = this.refresh$.pipe(
    switchMap(() => this.http.get<User>('/api/me')),
    shareReplay(1)
  );

  updateProfile(payload: Partial<User>) {
    return this.http.put('/api/me', payload).pipe(
      tap(() => this.refresh$.next()) // forces a refetch, updates every subscriber
    );
  }
}

```

```

// profile.component.ts
save() {
  this.userService.updateProfile(this.form.value).subscribe(() => {
    this.toast.success('Saved');
    this.router.navigate(['/dashboard']);
  });
}

```

For the "sometimes never catches up" case on slow connections: that's consistent with a race where the dashboard's own read of `user$` fires *before* the `tap(() => refresh$.next())` mutation completes its round trip, so if any other part of the app also reads `user$` independently and caches its own snapshot in a local field without staying subscribed, it can freeze on a stale emission. The fix above (subscribing to `user$` directly wherever it's displayed, rather than copying its value into a local field once) eliminates that too.

Verification: save a change, immediately navigate to dashboard and back repeatedly under throttled network, confirm the profile always shows the just-saved value with no stale flash; add an integration test asserting `UserService.user$` emits exactly once with fresh data per `updateProfile()` call.

Interviewer intent: Tests understanding of RxJS caching pitfalls (`shareReplay` without invalidation) and router component reuse — two entirely different root causes that produce an identical symptom, so the candidate must diagnose before assuming.

Scenario 9: The Modal That Won't Close (Sometimes)

Situation: A confirmation modal (built with a custom overlay service, not Angular Material) usually closes fine when clicking "Cancel" or the backdrop, but intermittently — maybe 1 in 20 times — it stays stuck open, unresponsive to any click, and the only way out is refreshing the page. QA can't reliably reproduce it on demand.

Question: How do you approach debugging an intermittent UI-stuck bug that can't be reliably reproduced?

Answer: Intermittent + can't reproduce on demand points toward a **timing/race condition**, most likely in how the modal's open/close state is being animated or how its destruction is sequenced relative to a CSS animation or an async close guard (e.g., "confirm you want to close" logic, or an exit animation `Promise` that sometimes never resolves).

Diagnosis steps:

1. Since it's not reliably reproducible by clicking normally, try reproducing by clicking rapidly / clicking close before the modal has fully finished opening (interrupting the open animation) — race conditions involving animation state are often triggered by acting mid-transition.
2. Check the close logic for reliance on an animation (`animation.done`) / `transitionend` event to actually remove the component from the DOM/overlay stack. If the close is gated on waiting for a CSS transition to end, and the user closes it *during* the opening transition (so no proper "closing" transition class is ever applied, or the transition is interrupted and `transitionend` never fires because the property never actually changed), the completion event that was supposed to trigger removal never comes, and the modal is stuck forever both visually and in the overlay stack (which is also why it becomes fully unresponsive — the stuck overlay may still be capturing pointer events even though visually it looks idle).
3. Confirm the overlay service tracks "is this the top-most modal accepting input" state correctly — if the close got interrupted, the overlay stack might think a *different* invisible layer is still on top, backdrop-blocking all clicks including on a completely different part of the page.

```
// BEFORE - closing depends entirely on a DOM transition event that may never fire
close() {
  this.el.classList.add('closing');
  this.el.addEventListener('transitionend', () => {
    this.overlayRef.dispose();
  }, { once: true });
}
```

If `close()` is called while the element is still mid-opening-transition (class `opening` still applied, `closing` added on top without removing `opening`), the browser may compute no net style change on the transitioned property in that frame, and `transitionend` simply never fires — the callback that disposes the overlay is never invoked, leaking the overlay and leaving it (or its backdrop) intercepting pointer events indefinitely.

Fix: never let disposal depend solely on a DOM event that isn't guaranteed to fire; always pair it with a timeout fallback, and make open/close state transitions mutually exclusive so an in-progress open can be cancelled cleanly:

```
close() {
  this.el.classList.remove('opening');
```

```

this.el.classList.add('closing');

const dispose = () => {
  this.el.removeEventListener('transitionend', onEnd);
  clearTimeout(fallback);
  this.overlayRef.dispose();
};

const onEnd = (e: TransitionEvent) => {
  if (e.target === this.el) dispose();
};

this.el.addEventListener('transitionend', onEnd);
// Guaranteed cleanup even if the transition never fires (interrupted, no-op, etc.)
const fallback = setTimeout(dispose, 300); // matches/exceeds CSS transition duration
}

```

Also worth adding: an escape hatch at the overlay-service level — a global "close all overlays" triggered by pressing Escape twice or a dev-only watchdog that flags any overlay still in the stack after N seconds past its close call, logged to the error monitor so this can be caught proactively rather than waiting for user complaints.

Verification: script an automated test that clicks close within 0-50ms of open across many trials (simulating the race) and assert the overlay is always disposed within 500ms; manually spam-click open/close in the real app for a few minutes to build confidence the timeout fallback covers all interrupted-transition cases.

Interviewer intent: Tests the candidate's approach to genuinely intermittent bugs — forming a hypothesis about *timing* rather than giving up at "can't reproduce," and knowing that DOM transition events are not a reliable sole trigger for cleanup logic.

Scenario 10: The Chart That Renders Blank Only for a Subset of Users

Situation: An analytics chart (rendered by a third-party charting library wrapped in an Angular component) shows correctly for most users, but a subset of users — seemingly all on corporate networks — see a completely blank chart area with no errors in their console. Your own testing on the corporate VPN reproduces it; testing on home wifi doesn't.

Question: How would you investigate a rendering bug that correlates with network environment rather than browser or OS?

Answer: Correlating with "corporate network" rather than browser/OS is the strong clue: this points to something being **blocked or altered at the network level** — a corporate proxy/firewall stripping something, a Content Security Policy difference enforced by corporate MDM browser policies, or the charting library trying to fetch a resource (web font, CDN script chunk) that's blocked by corporate web filtering, failing silently because the library swallows the load error instead of surfacing it.

Diagnosis steps:

1. On the VPN reproduction, check the Network tab specifically for any failed/blocked requests (red entries, `net::ERR_BLOCKED_BY_CLIENT` or similar) — many corporate proxies block requests to

ad/analytics-sounding domains or generic CDNs by category filter, and charting libraries sometimes lazy-load a sub-resource (a WASM binary, a web worker script, or a font) from a CDN rather than bundling it.

2. Check the Console for CSP violation reports — corporate-managed browsers sometimes enforce a stricter injected CSP via extension/policy that blocks inline `<style>` injection or `eval`-like usage, which some charting libraries use internally for dynamic style computation.
3. Check whether the chart depends on `ResizeObserver` or `IntersectionObserver` behaving normally — some corporate security extensions patch/throttle these APIs, and if the library waits for a `ResizeObserver` callback to know the container has a size before drawing, and that callback never fires (or fires with zero dimensions because the extension throttled/alterd it), the canvas ends up 0x0 and renders nothing, with no thrown error.

Root cause found: the charting library lazy-loads a Web Worker script from its own CDN for off-main-thread rendering, and the corporate proxy blocks requests to `*.workers.jsdelivr.net`-style hosts by category (script/CDN blocking policy), causing the worker to fail to instantiate. The library's own error handling swallows the worker construction failure and just never draws, rather than falling back to main-thread rendering or surfacing an error.

Fix: eliminate the dependency on an externally-fetched worker script by self-hosting all library assets (bundling the worker script through the build rather than fetching it from a CDN at runtime), and add an explicit render-failure fallback in the wrapper component so future silent failures are visible instead of invisible:

```
@Component({ selector: 'app-chart', ... })
export class ChartComponent implements AfterViewInit {
  protected renderFailed = signal(false);

  async ngAfterViewInit() {
    try {
      await this.chartLib.render(this.canvasRef.nativeElement, this.data(), {
        worker: false // force main-thread rendering, no runtime CDN dependency
      });
    } catch (err) {
      console.error('Chart render failed', err);
      this.renderFailed.set(true); // template shows a fallback + retry button
    }
  }
}
```

Verification: reproduce on the corporate VPN again after self-hosting assets and disabling the worker dependency; confirm the chart renders. As a longer-term policy, audit every third-party library in the app for runtime CDN fetches (fonts, workers, wasm) and self-host them, since corporate network variability is otherwise untestable in normal CI/staging environments.

Interviewer intent: Tests whether the candidate thinks about environment-dependent failures beyond "browser bugs" — network policy, CSP, and runtime asset loading are common real-world causes that don't show up in typical dev/staging testing.

Scenario 11: The Infinite Digest-Like Loop Freezing the Browser Tab

Situation: After adding a new "live validation summary" panel that shows a computed count of form errors next to each field, the entire page becomes unresponsive within a second of opening it — the tab shows "Page Unresponsive," and Chrome's own performance profiler shows a single call stack thousands of frames deep, all `checkAndUpdateView`-style Angular internals.

Question: What causes an Angular app to spiral into what looks like an infinite change-detection loop, and how do you fix it?

Answer: A call stack thousands of `checkAndUpdateView`-style frames deep is the classic signature of a change-detection loop: something in the template calls a function that **returns a new value/object every time it's evaluated**, so Angular's next CD cycle sees "something changed," schedules another check, evaluates the same function again, gets another new value, and repeats — this is one of the oldest and still most common Angular production bugs, usually introduced by calling a plain method directly in the template instead of a signal/computed or a pipe.

Diagnosis steps:

1. Search the new panel's template for any method calls in interpolation or bindings — e.g. `{{ getErrorCount() }}` or `[class.has-errors]="hasErrors()"` where the method does more than a trivial pure read.
2. Check if that method constructs a new array/object each call (e.g., `this.form.errors ? Object.keys(this.form.errors) : []` creating a new array reference every single time), which is harmless in isolation but becomes a loop if that new-reference result feeds into something that itself schedules another change detection pass — most dangerously, if it's used inside an `effect()` or a `computed()` that writes to a signal, or if `ChangeDetectorRef.detectChanges() / markForCheck()` is called from within a getter that's evaluated during CD, directly triggering re-entrant checks.

Root cause here: the panel component has an `effect()` that reads a method call (not a signal) as if it were reactive, and — critically — the effect's callback also *writes* to a signal that's used elsewhere in a way that re-triggers the same effect:

```
// BEFORE - effect reads a non-signal method result every run, and writes a signal
// that participates in what the method effectively depends on, causing a cycle
errorCount = signal(0);

constructor() {
  effect(() => {
    const count = this.computeErrorCount(); // plain method, not tracked, but re-runs anyway
    // due to detectChanges below
    this.errorCount.set(count);
    this.cdr.detectChanges(); // forces a synchronous CD cycle, which re-runs this effect's
    // dependents
  });
}

computeErrorCount() {
  return Object.keys(this.form.errors ?? {}).length;
}
```



```
}
```

Calling `detectChanges()` manually inside an `effect()` is the mechanism of the loop: it forces Angular to re-check the view synchronously, which re-evaluates any template expression depending on `errorCount()`, which (if anything downstream also happens to call something that further perturbs form state, like a validator recalculation) can re-fire the effect again before the call stack unwinds — explaining a stack thousands of frames deep instead of a normal bounded set of CD cycles.

Fix: never call `detectChanges()` / `markForCheck()` from within an `effect()`, and replace the manual "compute and set" pattern with a `computed()` signal that Angular manages declaratively with proper dependency tracking and no forced synchronous re-entry:

```
errorCount = computed(() => Object.keys(this.form.errors ?? {}).length);
```

```
<span>{{ errorCount() }} errors</span>
```

If the error count genuinely can't be derived as a pure signal (e.g., it depends on a non-reactive `FormGroup.errors` that doesn't emit signals natively), bridge it properly via `toSignal` on the form's `statusChanges` observable instead of a manual effect + `detectChanges`:

```
private status = toSignal(this.form.statusChanges, { initialValue: this.form.status });
errorCount = computed(() => {
  this.status(); // establishes the reactive dependency
  return Object.keys(this.form.errors ?? {}).length;
});
```

Verification: open the panel and confirm the tab stays responsive, profile with the Performance tab to confirm CD cycles are bounded (a handful per keystroke, not thousands), and add an ESLint rule/code-review checklist item flagging any `detectChanges()` / `markForCheck()` call inside an `effect()` body.

Interviewer intent: Tests deep understanding of Angular's change detection and the signals `effect()` API's re-entrancy hazards — specifically the anti-pattern of manually forcing CD from inside reactive primitives, which is a newer but increasingly common production bug as teams adopt signals.

Scenario 12: The Currency Field That Rounds Differently in Production Than in the Developer's Testing

Situation: Finance flags that invoice totals shown in the Angular app occasionally differ by a cent or two from the backend-calculated total, but only for specific customers (mostly in Germany and Japan). Developers testing locally (US locale machines) never see a discrepancy.

Question: Why would a numeric display bug correlate with customer locale, and how do you fix it properly?

Answer: This is a locale-and-floating-point interaction bug, and "developers on US locale machines never see it" is the giveaway — Angular's built-in `CurrencyPipe` / `DecimalPipe` (and `Intl` APIs generally) format numbers according to the **active locale** (`LOCALE_ID`), and different locales have different grouping/decimal separator and, more importantly, different **rounding behavior interacting with how the raw number was computed** if the underlying value itself is already an imprecise floating-point result (classic binary floating

point can't represent values like $0.1 + 0.2$ exactly), which becomes visible once formatted for display in a locale that rounds at a boundary differently, or — most commonly in real incidents — the app is doing its own manual currency math in JavaScript numbers instead of trusting the backend-provided total outright.

Diagnosis steps:

1. Confirm whether the discrepancy exists in the raw API response already, or is introduced client-side — log the exact raw JSON payload value for an affected German customer's invoice and compare it against what's rendered.
2. If the raw value matches the backend exactly but the *rendered* text differs from what Finance expects, the bug is in formatting/pipe configuration, not math — check `LOCALE_ID` registration and whether `CurrencyPipe` digit constraints (`'1.2-2'`) are being applied consistently, since some locales' default fraction digit rules differ subtly and an explicit format string is required to force consistency.
3. If the raw value itself is already off by a cent, the bug is upstream client-side math — check whether the invoice total is being *recomputed* client-side (e.g., summing line items with floating point addition) rather than displaying a value provided directly by the backend, and whether that recomputation uses `toFixed()` (which has its own well-known floating point rounding quirks, e.g. `(1.005).toFixed(2)` yielding `"1.00"` not `"1.01"`) instead of a proper decimal library.

Root cause found: the invoice component sums line-item totals client-side for display (`items.reduce((sum, i) => sum + i.price * i.qty, 0)`) instead of using the backend-provided, authoritative total, and pipes it through `CurrencyPipe` without the digit-info format string, so Japan (0 fraction digits typically for JPY) and Germany (comma as decimal separator, different default grouping) both expose rounding edge cases in that ad hoc sum that the US-locale developer's own test data never happened to hit because their test invoices' line items summed cleanly.

```
// BEFORE - client recomputes total via imprecise floating point math
get total() {
  return this.items.reduce((sum, i) => sum + i.price * i.qty, 0);
}
```

```
{{ total | currency:'EUR' }}
```

Fix: never recompute financial totals client-side for display when an authoritative backend value exists — trust and display the source of truth, and if client-side math is unavoidable (e.g., a live "estimated total" before save), use integer cents arithmetic or a decimal library instead of floating point:

```
// Preferred: display the backend's authoritative total directly.
protected total = computed(() => this.invoice().totalAmount); // already correct, from the
server
```

```
// If client-side estimation is genuinely required, do cents-integer math, not floats.
private estimateTotalCents = computed(() =>
  this.items().reduce((sum, i) => sum + Math.round(i.priceCents * i.qty), 0)
);
protected estimateTotal = computed(() => this.estimateTotalCents() / 100);
```

```
{{ estimateTotal() | currency: invoice().currencyCode : 'symbol' : '1.2-2' : locale }}
```

Also standardize the app's `LOCALE_ID` handling so the pipe's formatting is deterministic per customer, and add an explicit digit-info string rather than relying on locale defaults:

```
providers: [  
  { provide: LOCALE_ID, useFactory: () => currentUser.locale ?? 'en-US' }  
]
```

Verification: reproduce with a German-locale test account and the exact line items from a flagged invoice, confirm the displayed total now exactly matches the backend total to the cent; add unit tests asserting the display total equals `invoice.totalAmount` verbatim rather than any recomputation, across a matrix of locales (`en-US`, `de-DE`, `ja-JP`).

Interviewer intent: Tests awareness that floating-point arithmetic and `Intl` /locale formatting are real production correctness issues in financial UIs, and that the correct fix is usually "stop recomputing, trust the source of truth" rather than "add more rounding."

Scenario 13: The Autocomplete That Shows Results for the Previous Search Term

Situation: A location search autocomplete occasionally shows results that don't match what's currently typed in the box — e.g., user types "Berlin", quickly changes to "Boston", and the dropdown briefly (or persistently, on slower connections) shows Berlin, Germany results instead of Boston. It self-corrects if the user pauses.

Question: What RxJS-related race condition causes this, and what's the fix?

Answer: This is the canonical "out-of-order async response" race condition: multiple HTTP requests are in flight simultaneously (one for "Berlin", one for "Boston"), and because network latency is unpredictable, **the Berlin request's response can arrive after the Boston request's response**, overwriting the correct newer result with the stale older one. This happens because the search stream is very likely using `mergeMap` (or manually subscribing on every keystroke without cancellation) instead of an operator that cancels the previous in-flight request.

Diagnosis steps:

1. Confirm the operator used to flatten the search-term stream into HTTP calls — check for `mergeMap`, or worse, a bare `.subscribe()` inside a `(input)` event handler with no cancellation logic at all.
2. Reproduce reliably by throttling the network so response ordering can be controlled/observed, typing a term, quickly changing it, and watching the Network tab's response timing to confirm the earlier request does resolve after the later one.

```
// BEFORE - mergeMap runs every request concurrently and applies results in arrival order,  
// not request order, so a slow earlier response can overwrite a faster later one.  
this.searchTerm$.pipe(  
  debounceTime(200),  
  mergeMap(term => this.geoService.search(term))  
)  
.subscribe(results => this.results.set(results));
```

Fix: use `switchMap`, which cancels the previous inner observable (and, for `HttpClient`, actually aborts the underlying HTTP request) the moment a new search term arrives, guaranteeing only the latest request's response is ever applied:

```
private searchTerm = signal('');
private results$ = toObservable(this.searchTerm).pipe(
  debounceTime(200),
  distinctUntilChanged(),
  switchMap(term => term.length > 1
    ? this.geoService.search(term).pipe(catchError(() => of([])))
    : of([])
  )
);
protected results = toSignal(this.results$, { initialValue: [] });
```

It's worth explaining to an interviewer *why* `switchMap` is correct here and `mergeMap` / `concatMap` are not: `mergeMap` runs all requests concurrently with no ordering guarantee on completion (today's bug); `concatMap` would queue requests strictly in order, which fixes the overwrite problem but reintroduces a different bug — the UI would wait for Berlin's slow response to finish before even starting the Boston request, making the box feel laggy. `switchMap` is correct specifically because stale in-flight requests should be abandoned entirely, not queued, when a newer input supersedes them.

Verification: throttle the network so the first request is guaranteed to resolve after the second is fired, type "Berlin" then quickly "Boston", and confirm only Boston results ever appear, and confirm via the Network tab that the Berlin request shows as cancelled (red, "(cancelled)") once the newer request is issued.

Interviewer intent: Tests whether the candidate can articulate the precise difference between `switchMap`, `mergeMap`, `concatMap`, and `exhaustMap` in terms of real production consequences, not just recite definitions.

Scenario 14: The App That Crashes Only When a User's Browser Back Button Is Spammed

Situation: Bug reports (mostly from power users) describe the app going to a completely blank white screen with a console error about `Cannot read properties of undefined (reading 'subscribe')` after rapidly clicking the browser's back button several times in a row while on a data-heavy detail page.

Question: Diagnose a crash that only happens under rapid back-navigation.

Answer: Rapid back-navigation is a strong trigger for **route parameter/data resolution races combined with improperly guarded lifecycle cleanup** — specifically, a component subscribing to `ActivatedRoute.params / data` and using the emitted value to call a service method, where rapid navigation causes the component to be destroyed and a *new* instance created (or the same instance reused, receiving multiple rapid emissions) faster than an in-flight async chain can complete, and something downstream in that chain is dereferencing a property on a value that's now `undefined` because it belonged to the previous, already-torn-down navigation's context.

Diagnosis steps:

1. Reproduce with browser back-button spam while watching the console for the exact error and stack trace; identify which subscription's callback the crash originates from.

2. Check whether the component reads `ActivatedRoute.snapshot` inside an async callback (e.g., inside a `.subscribe()` that resolves after navigation has already moved on) — `snapshot` at call time inside a delayed callback reflects **whichever route is currently active**, not the route the observable chain started from, so if the route has already changed (or the resolved data was cleared) by the time the callback fires, properties expected from the old page's snapshot are now `undefined`.
3. Check if there's a manual subscription in `ngOnInit` that doesn't use `takeUntilDestroyed()` / `async` pipe, so on rapid navigation, a stale subscription from a torn-down component instance keeps running and eventually calls into `this.someProperty.subscribe(...)` where `this.someProperty` was itself set to `undefined` in `ngOnDestroy` cleanup that ran while the async chain was still in flight.

```
// BEFORE
ngOnInit() {
  this.route.params.subscribe(params => {
    this.itemService.getDetail(params['id']).subscribe(detail => {
      // by the time this resolves, the user may have navigated away twice already;
      // `this.relatedService` may already be undefined if ngOnDestroy nulled it out
      this.relatedService.subscribe(r => this.related.set(r));
    });
  });
}

ngOnDestroy() {
  this.relatedService = undefined as any; // manual "cleanup" that races with the above
}
```

Fix: stop manually nulling out fields for "cleanup" (that's what subscription teardown is for, not property nulling) and ensure every subscription chain — including nested ones — is tied to the component's destruction via `takeUntilDestroyed()`, so an in-flight chain simply never calls its callback after the component is gone rather than calling it and hitting `undefined`:

```
export class DetailComponent {
  private route = inject(ActivatedRoute);
  private itemService = inject(ItemService);
  private destroyRef = inject(DestroyRef);

  protected related = signal<Related[]>([]);

  constructor() {
    this.route.params.pipe(
      switchMap(params => this.itemService.getDetail(params['id'])),
      switchMap(detail => this.itemService.getRelated(detail.id)),
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(related => this.related.set(related));
  }
}
```

Two things fixed this simultaneously: chaining with `switchMap` instead of nested `.subscribe()` calls means a new route param automatically cancels the prior in-flight detail+related lookup instead of letting stale chains keep running; and `takeUntilDestroyed()` guarantees that if the component itself is destroyed mid-chain (rapid navigation swapping route components entirely), the callback never fires at all against a torn-down instance.

Verification: script an automated rapid back/forward navigation test (10+ clicks in under a second) across the detail page and confirm no console errors and no blank screen; manually stress test on the actual power-user hardware profile if available (older machines exaggerate the race window).

Interviewer intent: Tests whether the candidate defaults to proper reactive composition (`switchMap` chains + `takeUntilDestroyed`) instead of nested subscriptions with manual field nulling — a structural anti-pattern that causes exactly this class of navigation race crash.

Scenario 15: The Silent Failure Where Form Validation Passes But Bad Data Reaches the Server

Situation: A support ticket reveals a customer's shipping address was saved with an empty postal code, even though the form field is marked required with a red asterisk and the form appeared to block submission for other users. It's not consistently reproducible — most attempts to submit an empty postal code are correctly blocked.

Question: How would you track down validation that "usually" works but occasionally lets bad data through?

Answer: "Usually works, occasionally doesn't" for a *required field* check is a strong signal of a validator being **attached asynchronously or conditionally** in a way that has a timing gap — most commonly, the form control's validators are set up in response to something (like a country selection changing which fields are required) and there's a window where the control temporarily has no validators at all, during which a fast/scripted submit (or a user who tabs through very quickly) can slip through before the validator attaches.

Diagnosis steps:

1. Check whether the postal code field's `validators.required` is applied unconditionally at form construction, or added/removed dynamically based on another field (e.g., country) via `setValidators()` / `clearValidators()` triggered by a `valueChanges` subscription.
2. If dynamic, check whether `updateValueAndValidity()` is called synchronously right after `setValidators()`, and whether there's any `async` gap (e.g., `setTimeout`, a debounce, or an HTTP call to fetch country-specific field rules) between the country changing and the postal code validator actually being (re)attached — if the user can click Submit during that gap, `form.valid` may evaluate `true` because the control's validator array is momentarily empty.

```
// BEFORE – validator update is debounced/async, leaving a window where the
// control temporarily has no required validator at all
this.form.get('country')!.valueChanges.pipe(
  debounceTime(300) // meant to avoid thrashing on typeahead country selection
).subscribe(country => {
  const postal = this.form.get('postalCode')!;
  if (countryRequiresPostalCode(country)) {
    postal.setValidators([validators.required]);
  } else {
    postal.clearValidators();
  }
  postal.updateValueAndValidity();
});
```

If a user selects their country and clicks Submit within that 300ms debounce window (very plausible for someone tabbing through a form quickly, or for any automated/keyboard-driven fast fill), `form.valid` is checked against the *previous* validator state, which — depending on the form's initial construction — might have had no validator on `postalCode` yet at all (e.g., if the field starts with no validators until the first country change ever fires).

Fix: two changes — first, never gate `validators.required` correctness on a debounced/async side channel; validity should be derivable synchronously from current form state. Second, defense in depth: also validate server-side (which should already exist, but this incident is exactly why it's not optional) so a client-side timing bug can never actually persist bad data:

```
// Compute required-ness synchronously as part of a cross-field validator
// on the group itself, not via a debounced side-effect subscription.
private form = new FormGroup({
  country: new FormControl(''),
  postalCode: new FormControl(''),
}, { validators: postalCodeRequiredForCountry });

function postalCodeRequiredForCountry(group: AbstractControl): ValidationErrors | null {
  const country = group.get('country')!.value;
  const postal = group.get('postalCode')!.value;
  if (countryRequiresPostalCode(country) && !postal) {
    return { postalCodeRequired: true };
  }
  return null;
}
```

```
// server-side (defense in depth, non-Angular but essential):
// reject the write outright if postalCode is missing for a country that requires it,
// regardless of what the client claimed was valid.
```

Verification: add a unit test that changes country and calls `form.valid` **synchronously in the same tick**, asserting the postal code requirement is reflected immediately with no timing dependency; add an E2E test that rapidly selects country then clicks submit with no pause, confirming submission is blocked every time. Also add a server-side integration test with a deliberately malformed payload (no client involved at all) to

confirm the backend independently rejects it — closing the loop so this can never silently persist bad data again even if a future client bug reintroduces a timing gap.

Interviewer intent: Tests whether the candidate recognizes that "sometimes passes validation" points to asynchronous/debounced validator attachment rather than a logic bug in the validator itself, and whether they instinctively reach for server-side validation as a non-negotiable safety net rather than treating client validation as sufficient.

Scenario 16: The Page That Works for QA But Every Real User Reports It "Sometimes Doesn't Load"

Situation: QA, using company laptops on the corporate network, can never reproduce a bug where the app gets stuck on the loading spinner forever. Real users on home connections report it a few times a week, always on first load of the day (subsequent loads that day are fine). Clearing cache "fixes" it for a while.

Question: Why would "clearing cache fixes it temporarily" be such an important clue, and what's the likely bug?

Answer: "Clearing cache fixes it, only first load of the day, QA on fast reliable networks never sees it" points squarely at a **service worker / build-versioning cache problem**: the app almost certainly uses Angular's `@angular/service-worker` for PWA caching, and users are getting stuck because the service worker served a **stale cached `index.html` /app shell that references JS chunk hashes from a previous deployment**, while the actual chunk files at those old hashes have since been deleted from the server (because a new deployment replaced them) — so the browser requests `main.a1b2c3.js`, gets a 404, and the app never finishes bootstrapping, leaving the spinner forever. This disproportionately hits real users because they leave tabs/browsers closed overnight and revisit after a deploy happened, while QA constantly hard-refreshes or uses fresh sessions that pick up the current deployment cleanly.

Diagnosis steps:

1. Check `Application` tab → Service Workers in DevTools for an affected repro (can be forced by throttling or manually deploying a new version and then loading an old cached tab) — look for a service worker in "waiting" or serving stale content.
2. Check the Network tab for 404s on chunk files referenced by the cached `index.html` immediately after the spinner appears — this confirms hash mismatch between the cached shell and currently-deployed assets.
3. Confirm the deployment process: does it delete old build artifacts immediately on deploy, or keep a rolling window of recent builds available? Immediate deletion combined with a service worker that hasn't yet updated is the exact combination that produces this.

Root cause: the `ngsw-config.json` was set up correctly for asset caching, but the app doesn't handle the `swUpdate` "new version available" event at all — the service worker dutifully detects updates in the background per its normal lifecycle, but without the app explicitly prompting an activation/reload, a user can be sitting on an old, now-broken cached version for an extended idle period (overnight) if they don't reload in the meantime, especially compounded by a deploy pipeline that doesn't retain previous build artifacts for the safety window the service worker's own update lifecycle assumes.

```
// BEFORE – no swUpdate handling at all; relies purely on the SW's default lifecycle
// and the user happening to do a full reload at the right time
```


Fix: explicitly listen for available updates and force activation + reload proactively, and adjust the deployment pipeline to retain the previous build's assets for a safety window:

```
@Injectable({ providedIn: 'root' })
export class UpdateService {
  private swUpdate = inject(SwUpdate);

  constructor() {
    if (!this.swUpdate.isEnabled) return;

    this.swUpdate.versionUpdates.pipe(
      filter((evt): evt is VersionReadyEvent => evt.type === 'VERSION_READY')
    ).subscribe(() => {
      // Activate the new version and reload, rather than leaving the user
      // on a shell that will eventually reference deleted chunk files.
      this.swUpdate.activateUpdate().then(() => document.location.reload());
    });

    // Also proactively check for updates on app start / periodically,
    // not just relying on the SW's own background check timing.
    this.swUpdate.checkForUpdate();
  }

  // Catch the "unrecoverable state" case directly – this fires precisely
  // when the SW detects it can't reconcile the cached shell with reality
  // (e.g., a chunk 404), which is exactly this bug's failure mode.
  handleUnrecoverable() {
    this.swUpdate.unrecoverable.subscribe(() => {
      document.location.reload();
    });
  }
}
```

```
// deployment pipeline: retain at least the previous 1-2 build's asset directories
// (or keep them versioned/immutable in a CDN, never deleted immediately),
// so any client still on a slightly-stale cached shell can still fetch its chunks
// until it naturally updates.
```

Verification: deploy a new version, then from an already-open tab on the old version, simulate the update flow and confirm the app auto-reloads onto the new version instead of getting stuck; also confirm the pipeline change by checking that assets from N-1 deployments remain fetchable for at least 24-48 hours post-deploy. Add monitoring/alerting on 404 rates for chunk requests specifically, since a spike there is the earliest signal of this exact failure mode recurring.

Interviewer intent: Tests whether the candidate understands Angular service worker update lifecycle and PWA caching pitfalls in production — a bug class that's invisible in typical QA/dev workflows and only surfaces from real deployment cadence interacting with real user idle patterns.

Scenario 17: The Grid That Shows the Right Data in the Wrong Rows After Sorting

Situation: A data grid lets users click a column header to sort. After sorting by "Amount," the amount column values are correctly sorted, but other columns (Customer Name, Date) in the same rows no longer match — e.g., a row shows the highest amount but a customer name and date that belonged to a completely different row before sorting.

Question: How would you diagnose data that gets "shuffled" incorrectly across columns during a sort operation?

Answer: This is a classic symptom of sorting **one array while rendering from a different, now-desynchronized array or index mapping** — most often, the sort function mutates or reorders only part of the row's data (e.g., sorting an array of raw `amount` values separately from the array of full row objects), or the grid maintains parallel arrays (one per column) instead of an array of row objects, and the sort only reordered one of the parallel arrays.

Diagnosis steps:

1. Inspect the sort implementation directly: is it sorting `this.rows` (an array of complete row objects) in place, or is it sorting a derived/extracted array (e.g., `this.rows.map(r => r.amount).sort(...)`) and then trying to re-zip it back together with other still-original-order arrays?
2. Check if the grid uses `@for` with `track $index` (again, index-based tracking) on a *rendering* structure that's separate from the actual data model — if column cells are rendered from separate bound arrays per column rather than one row object per iteration, a sort that only reorders the "amount" array desyncs instantly from the untouched "name"/"date" arrays.

Root cause: the component keeps parallel arrays instead of one array of row objects (a legacy pattern, often left over from a CSV-parsing implementation that built columns independently), and the sort handler only reorders the `amounts` array being displayed, using its sorted indices to reorder... nothing else:

```
// BEFORE - parallel arrays, sort desyncs everything else
customerNames: string[] = [];
amounts: number[] = [];
dates: Date[] = [];

sortByAmount() {
  this.amounts = [...this.amounts].sort((a, b) => b - a);
  // customerNames and dates were never touched - now misaligned with amounts
}
```

Fix: model the grid's data as a single array of row objects — the only representation where "sort" can never desynchronize columns, because there's nothing to desynchronize; each row carries all its fields together through any reordering:

```
interface Row { id: string; customerName: string; amount: number; date: Date; }

protected rows = signal<Row[]>([...]);
protected sortKey = signal<keyof Row>('amount');
protected sortDir = signal<'asc' | 'desc'>('desc');

protected sortedRows = computed(() => {
  const key = this.sortKey();
  const dir = this.sortDir() === 'asc' ? 1 : -1;
  return [...this.rows()].sort((a, b) => (a[key] > b[key] ? 1 : -1) * dir);
});
```

```
@for (row of sortedRows(); track row.id) {
  <tr>
    <td>{{ row.customerName }}</td>
    <td>{{ row.amount | currency }}</td>
    <td>{{ row.date | date }}</td>
  </tr>
}
```

Verification: sort by every column in both directions and manually spot-check several rows against the pre-sort source data (e.g., the raw API response) to confirm every field in a row still belongs together after sorting; add a unit test asserting that for every row, `sortedRows().find(r => r.id === original.id)` has fields deep-equal to the original unsorted row's fields, for all sort configurations.

Interviewer intent: Tests whether the candidate recognizes parallel-array data modeling as an anti-pattern that causes exactly this class of desync bug, and defaults to row-object-per-record modeling with `track` by stable id.

Scenario 18: The Environment Variable That "Leaks" the Wrong API URL in Production

Situation: After a deployment, some users briefly see failed API calls pointed at `https://staging-api.internal.example.com` instead of the production API, even though the deployed build was built with `--configuration production`. It resolves itself for everyone within about 10-15 minutes without any redeploy.

Question: How can a production build end up calling a staging URL, and why would it self-resolve without a redeploy?

Answer: "Self-resolves in 10-15 minutes without a redeploy" rules out a build configuration mistake (that would affect 100% of users permanently until the next deploy) and instead points to a **CDN/edge cache serving a stale, previously-deployed bundle** to a subset of users — most likely a previous deployment (perhaps a preview/staging deploy that was live briefly, or a canary) got cached at the CDN edge with a longer TTL than intended, and different edge nodes/regions serve users different cached versions until the cache naturally expires (which matches "self-resolves after ~10-15 minutes," consistent with a short-but-nonzero CDN TTL).

Diagnosis steps:

1. Check response headers (`Cache-Control`, `Age`, `X-Cache` / `CF-Cache-Status` or equivalent) on the

actual served `index.html` /main bundle for an affected user session — an `Age` header showing the resource has been cached for exactly the observed incident window, or `X-Cache: HIT` from an edge node, confirms CDN-level staleness rather than an app code bug.

2. Compare the actual JS bundle content being served (view source / fetch the exact chunk URL) against what was just deployed — if it matches an *older* build's environment config, this is entirely a deployment/caching pipeline issue, not an Angular code defect.
3. Check the deploy pipeline for how `environment.prod.ts` / `fileReplacements` are applied and whether a staging build artifact was ever accidentally promoted or deployed to the production CDN path (e.g., a build script race where a staging deploy and production deploy overlapped, or a shared CDN path briefly served the wrong bucket's contents during a deploy).

While this is largely an infra/deploy-pipeline issue rather than a pure Angular code bug, the Angular-relevant fixes are: ensure builds are fully immutable and content-hashed (already default via `outputHashing: "all"`) so that even if an old `index.html` is served briefly, at minimum the JS/CSS chunk URLs it references are guaranteed to still exist (never overwritten in place), and ensure `environment.ts` / `environment.prod.ts` values are correctly wired via Angular's `fileReplacements` so there's no ambiguity in which config compiled into which bundle:

```
// angular.json – confirm fileReplacements are configured for the production configuration
"configurations": {
  "production": {
    "fileReplacements": [
      { "replace": "src/environments/environment.ts", "with":
"src/environments/environment.prod.ts" }
    ],
    "outputHashing": "all"
  }
}
```

```
// environment.prod.ts – confirm this is genuinely committed and correct,
// not accidentally identical to a staging file due to a bad merge
export const environment = {
  production: true,
  apiUrl: 'https://api.example.com',
};
```

The actual operational fix: set `Cache-Control: no-cache` (or a very short max-age with `must-revalidate`) on `index.html` specifically at the CDN, since `index.html` is the one file that must always be fetched fresh (it's the entry point referencing content-hashed chunk names); long-lived caching should be reserved only for the hashed JS/CSS assets themselves, which are safe to cache aggressively since their filename changes whenever content changes:

```
# CDN config
/index.html      Cache-Control: no-cache, must-revalidate
/*.<hash>.js     Cache-Control: public, max-age=31536000, immutable
```

Verification: after correcting the CDN cache headers, deploy a new build and confirm via response headers that `index.html` is always revalidated (never served from a stale edge cache), while hashed chunks remain cached long-term for performance; monitor for any recurrence by alerting on API calls hitting the staging domain from production traffic (a canary metric that would have caught this incident immediately).

Interviewer intent: Tests whether the candidate can reason beyond application code into the deployment/CDN layer when a bug's timing signature (self-resolves without a redeploy) rules out a pure code defect — a skill senior engineers need even though it's adjacent to, not strictly inside, Angular itself.

Scenario 19: The Component That Renders Twice, Causing Duplicate Analytics Events

Situation: The analytics team reports that every "page viewed" event is being logged exactly twice for logged-in users, but only once for anonymous/logged-out users. The event is fired from `ngOnInit` of a shared layout component that wraps all authenticated routes.

Question: Why would a component's `ngOnInit` run twice only for one class of users, and how do you fix the duplicate firing?

Answer: "Only for logged-in users" is the key clue — it means whatever is causing the double-init is tied to something in the authenticated code path specifically, most likely an **auth guard that itself causes an extra navigation/component (re)creation cycle**, or — very commonly — an `APP_INITIALIZER` /auth-resolution flow that briefly renders the authenticated shell once optimistically, then re-renders it again once the real auth state resolves, because the guard or a parent route's resolver re-triggers component creation rather than just gating data.

Diagnosis steps:

1. Add a unique instance id logged in the constructor (not just `ngOnInit`) of the layout component and check the console for logged-in vs logged-out sessions — if two distinct instance ids appear for logged-in users, the component is genuinely being created twice (not just `ngOnInit` running twice on the same instance, which would be unusual on its own and point to a different, rarer bug around manual re-invocation).
2. Check the router configuration for authenticated routes: is there a redirect happening as part of login (e.g., `/dashboard` guard redirects unauthenticated users to `/login`, but on *successful* auth check, some implementations navigate to the *same* target route again via `router.navigate()` rather than simply allowing the guard's boolean/`UrlTree` return to let the original navigation proceed) — this double-navigation is a very common real-world bug where a guard both returns `true` *and* imperatively calls `router.navigateByUrl()` as a leftover from an older imperative auth flow, causing the destination component to mount once from the original navigation and again from the guard's redundant explicit navigation.

```
// BEFORE – guard both allows navigation AND imperatively re-navigates,
// causing the target route/component to be processed twice
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    router.navigateByUrl(state.url); // redundant – navigation is already in progress
    return true;
  }
  router.navigateByUrl('/login');
  return false;
};
```

Fix: a guard should only ever return `true`, `false`, or a `UrlTree` — never *also* imperatively navigate when allowing the current navigation to proceed, since returning `true` alone is sufficient and the redundant explicit navigation is what creates the duplicate render pass:

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true; // let the in-progress navigation complete normally, no re-navigation
  }
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
};
```

As an additional safety net regardless of the root cause, move the analytics firing itself to a place that's inherently resistant to duplicate component creation — a router-level `NavigationEnd` subscription in a single app-wide service is more robust for page-view tracking than firing from every layout component's `ngOnInit`, since it ties the event to one authoritative navigation lifecycle signal rather than component instantiation count:

```
@Injectable({ providedIn: 'root' })
export class AnalyticsService {
  constructor() {
    const router = inject(Router);
    router.events.pipe(
      filter((e): e is NavigationEnd => e instanceof NavigationEnd),
      distinctUntilChanged((a, b) => a.urlAfterRedirects === b.urlAfterRedirects)
    ).subscribe(e => this.trackPageView(e.urlAfterRedirects));
  }
}
```

Verification: log in and navigate through several authenticated pages, confirming exactly one analytics event fires per navigation via a network/log inspection; remove the redundant guard navigation and confirm the layout component's constructor now logs only a single instance id per navigation for logged-in users, matching the logged-out behavior.

Interviewer intent: Tests knowledge of Angular route guard semantics (return value vs. imperative navigation) and whether the candidate reaches for a structurally more robust analytics-firing pattern (router events) rather than patching the symptom with a "hasFired" flag.

Scenario 20: The Feature That Breaks Only for Users with Browser Extensions Installed (Ad Blockers)

Situation: A small percentage of users (later found to correlate heavily with ad-blocker usage) report that a "Recommended Products" widget on the product page never appears, and the console shows a `net::ERR_BLOCKED_BY_CLIENT` for a request, but the rest of the page functions normally — until users then also report that submitting the "Add to Cart" form on that same page silently does nothing for them either, which seems unrelated to a blocked recommendations widget.

Question: How are these two seemingly unrelated symptoms (blocked recommendations widget, broken Add to Cart) connected, and what's the fix?

Answer: The connecting thread is that ad blockers work by pattern-matching request URLs (and sometimes element class names/ids) against block-lists, and the "Recommended Products" endpoint likely has a URL or parameter naming that superficially resembles a tracking/ads endpoint (e.g., `/api/recommendations?tracking_id=...` or a path segment like `/ads/` used generically for "advertised/recommended" products) — that request gets blocked outright by the extension, which is a relatively "expected" failure mode, but the *unrelated-looking* Add to Cart failure reveals the real production bug: an **unhandled promise rejection / uncaught RxJS error from the blocked request is bubbling up and breaking the shared page-level error boundary or a shared component state**, taking down functionality that has nothing to do with recommendations.

Diagnosis steps:

1. Reproduce with an ad blocker extension enabled locally; confirm the recommendations request is blocked and check the console for the resulting error — specifically whether it's an *uncaught* error (no `catchError` in the observable chain, or a `fetch()` call with no `.catch()`), and whether that uncaught error originates from code that shares scope/state with the Add to Cart button (e.g., both are part of the same parent component, and the parent's `ngOnInit` does `forkJoin([recommendations$, cartState$])` — if `forkJoin` sees *any* source error, it immediately errors out entirely and never emits, and if `cartState$` was combined into that same `forkJoin`, an error from the blocked recommendations call means the cart initialization data the Add to Cart button depends on never arrives either.

```
// BEFORE – forkJoin fails entirely if any single source errors,
// taking down unrelated cart initialization along with the blocked recommendations call
ngOnInit() {
  forkJoin([
    this.recommendationsService.getRecommendations(this.productId),
    this.cartService.getCartState()
  ]).subscribe(([recommendations, cart]) => {
    this.recommendations.set(recommendations);
    this.cartState.set(cart); // never runs if recommendations errored – breaks Add to Cart
  });
}
```

This is the actual mechanism: `forkJoin` requires *every* source to complete successfully before emitting anything; a blocked/errored recommendations request means `cartState` never gets set, and the Add to Cart button's logic (which depends on `cartState()` being populated) silently no-ops because it's checking a signal that was simply never initialized — with no error visibly thrown at the Add to Cart call site itself, making the two symptoms look unrelated when they share one root cause.

Fix: isolate independent data sources so a failure in one (especially one prone to third-party blocking, like recommendations/analytics-adjacent calls) can never cascade into unrelated functionality — catch errors per-source, not for the combined operation, and rename the endpoint away from ad-blocker-triggering patterns as a pragmatic mitigation for the recommendations feature itself:

```
ngOnInit() {
  this.cartService.getCartState()
    .pipe(catchError(() => of(this.cartService.emptyCartState())))
    .subscribe(cart => this.cartState.set(cart));

  this.recommendationsService.getRecommendations(this.productId)
    .pipe(catchError(() => {
      console.warn('Recommendations unavailable (possibly blocked by client)');
      return of([]); // widget simply doesn't render, everything else unaffected
    }))
    .subscribe(recs => this.recommendations.set(recs));
}
```

```
// Pragmatic mitigation for the recommendations endpoint itself: rename away from
// ad-blocker-triggering path/param patterns (avoid "ads", "tracking_id", "sponsored", etc.
// in the URL) so the feature isn't needlessly blocked in the first place.
getRecommendations(productId: string) {
  return this.http.get<Product[]>(`/api/products/${productId}/suggested-items`);
}
```

Verification: with an ad blocker enabled, confirm the recommendations widget gracefully omits itself (or shows a lightweight fallback) while Add to Cart and all other page functionality work identically to a no-ad-blocker session; add an application-wide principle (and lint/code-review checklist item) that `forkJoin` / `combineLatest` should never combine an "optional/best-effort" data source with a "required for core functionality" one without per-source error handling.

Interviewer intent: Tests whether the candidate can trace two superficially unrelated symptoms back to a shared RxJS combination-operator root cause, and understands that resilience to partial third-party/network failures requires isolating error handling per source rather than combining everything into one all-or-nothing operator.

Quick Revision Cheat Sheet

- **"Stopped updating" is a change-detection question first.** Confirm whether data is even reaching the handler before suspecting zones; shared observables (`shareReplay` without `catchError`) can silently die from one bad emission and freeze every subscriber.
- **Memory leaks that grow with tab lifetime are almost always subscriptions to long-lived**

services/singletons that outlive the component. Always tie subscriptions to `takeUntilDestroyed()`; verify with heap snapshot comparisons, not guesses.

- **Double-submit race conditions need defense in depth:** disable the control during the in-flight request, never apply blanket `retry()` to non-idempotent HTTP methods, and use a server-side idempotency key as the real safety net.
- **"Data silently lost" often means a service was provided at the wrong DI scope,** so a "shared" state service is actually being re-instantiated per route/component, discarding state on every navigation.
- **Visual flicker with correct data is a track/reconciliation problem.** Always track `@for` by stable entity id, never by `$index` on a list that reorders or filters, and avoid cloning objects unnecessarily in derived signals.
- **Bugs that only reproduce in production builds usually trace to minification/property-renaming, `instanceof/.name` checks across bundle boundaries, or staging not actually running a production build config.** Always enable production source maps.
- **Third-party scripts can break Angular via Zone.js API patch conflicts or capture-phase event listeners that call `stopPropagation()` globally.** Sandbox untrusted embeds in iframes when possible.
- **Stale data after navigation usually means an HTTP cache (`shareReplay`) was never invalidated on mutation, or a route-reuse strategy prevented `ngOnInit` from re-running.** Diagnose by checking if `ngOnInit` even fires again.
- **Intermittent "stuck" UI (modals, overlays) is a timing/race condition, most often tied to relying solely on a DOM event (`transitionend`) that isn't guaranteed to fire.** Always pair event-based cleanup with a timeout fallback.
- **Environment-correlated bugs (corporate network, ad blockers, specific locales) point outside pure app logic** — CSP, proxy filtering, CDN blocklists, and `Intl` locale rules are common real causes invisible in typical dev/QA environments.
- **`effect()` that calls `detectChanges()` / `markForCheck()` on itself is a common source of new infinite-loop bugs under signals** — let `computed()` and proper dependency tracking do the work instead of manual synchronous re-triggering.
- **Never recompute authoritative values (currency totals, cart state) client-side when a backend source of truth exists** — floating point and locale formatting differences are real production correctness bugs, not theoretical ones.
- **`switchMap` cancels stale in-flight requests; `mergeMap` doesn't.** Out-of-order async responses ("shows results for the previous search") are almost always a `mergeMap`-where-`switchMap`-belongs bug.
- **Validators attached asynchronously/conditionally (via debounced `valueChanges`) can leave a timing window where a required field has no validator at all.** Compute cross-field validity synchronously, and always back client validation with server-side validation.
- **Parallel arrays (one per column) instead of one array of row objects will desynchronize on any sort/filter/reorder.** Model rows as objects; track by id.
- **`forkJoin` / `combineLatest` fail entirely if any single source errors** — never combine an optional/best-effort source with one that's required for core functionality without per-source `catchError`.
- **Route guards should return `true` / `false` / `UrlTree` only** — never also imperatively

`router.navigate()` **when allowing a navigation to proceed**, or the target component/effects can run twice.

- **When a bug's timing signature rules out a code defect** (self-resolves without a redeploy, correlates with network/CDN edge behavior), look at the deployment and caching layer, not the Angular source.
- **Always reproduce with the same build configuration, network conditions, and environment (locale, extensions, corporate proxy) the affected users actually have** — "works on my machine" for a production bug usually means the repro environment differs in exactly the dimension that matters.

Created By - Durgesh Singh